SHELL PROGRAMMING : SHELL SCRIPT WRITING AND EXECUTING PARAMETERS & VARIABES , KEY-WORD VARIABLES

Steps to write and execute a script

- Open the terminal. Go to the directory where you want to create your script.
- Create a file with .sh extension.
- Write the script in the file using an editor.
- Make the script executable with command chmod +x <fileName> or chmod –R 777 .
- Run the script using ./<fileName>.

Note: In the last step you have to mention the path of the script if your script is in other directory. First of all, create a simple script in any editor or with echo. Then we'll make it executable with chmod +x command. To find the script you have to type the script path for the shell.

$ vi code1.sh
Echo "Shell Programming [Press i for insert mode

Press Esc Key

:wq - save & exit

$ chmod –R 777 .

Running the shell script :
./code1.sh
Output
Shell Programming
Shell Script to Demonstrate Special Parameters
Here, we are going to see what are the special Parameters of the shell script. Before that first, let's understand what parameters in the shell is. The parameter is the entity that stores the value. The variables are the parameters that are defined by the user to use in that specific shell script. And the Special parameters are the read-only variables that are predefined and maintained by the shell. Now let's see what are the Special parameters in the shell script.

S.No	Special Parameters	Description
1	$#	This parameter represents the number of arguments passed to the shell script.
2	$0	This parameter represents the script name.
3	$i	This parameter represents the ith argument passed to the shell script like

S.No	Special Parameters	Description
		$1,$2
4	$*	This parameter gives all arguments passed to the shell script separated by the space.
5	$!	This parameter gives PID of the last background running process.
6	$?	This parameter represents the exit status of the last command that executed. The 0 code represents success and 1 represents failure.
7	$_	This parameter gives the last argument provided to the previous command that executed.
8	$$	This parameter gives the PID of the current shell.
9	$@	This parameter holds all argument passed to the script and treat them as an array. It is similar to the $* parameter
10	$-	This parameter represents the current flags set in your shell .himBH are the flags in bash shell. Where: • H – histexpand • m – monitor • h – hashall • B – braceexpand • i – interactive

Now let's see the script which demonstrates all Special Parameters.

```
echo "Number of argument passed: $#"
echo "Script name is $0"
echo "The 2nd argument passed is: $2"
echo "Arguments passed to script are: $*"
echo "Exit status of last command that executed:$?" #This is the previous command for $_
echo "Last argument provide to previous command:$_"
echo "PID of current shell is: $$"
echo "Flags are set in the shell: $-"
```

Shell Script Parameters

Shift command is a built-in command. Command takes number as argument. Arguments shift down by this number.

For example, if number is 5, then $5 become $1, $6 become $2 and so on.
Example:
The shift command is mostly used when arguments are unknown. Arguments are processed in a while loop with a condition of (($#)). this condition holds true as long as arguments are not zero. Number of arguments are reduced each time as the shift command executes.

```
sssit@JavaTpoint: ~
#!/bin/bash

if [ "$#" == "0" ]
 then
  echo pass at least one parameter.
  exit 1
fi

while (( $# ))
 do
   echo you gave me $1
   shift
 done
```

Look at the above snapshot, this is our script.

```
sssit@JavaTpoint: ~
sssit@JavaTpoint:~$ ./xyz
pass at least one parameter.
sssit@JavaTpoint:~$ ./xyz hello all
you gave me hello
you gave me all
sssit@JavaTpoint:~$
```

Variables in Linux

Variables are an important part of any program or script.a variable is an element to refer block of data in memory that can be modified.a linux or unix variable can be assigned any type of value,such as text string or number.in linux there are two type of variables.

System Variable

An environment variable is a variable that is available to any child process of shell.usually,these variables are defined and needed by the programs that it runs, it is created and maintained by the Linux itself. This type of variable defined in Capital letters. Some of the system variables are shown below.

Variable	Example	Descriptions
BASH_VERSION	echo $BASH_VERSION	Holds the version of this instance of bash.

Variable	Example	Descriptions
HOSTNAME	echo $HOSTNAME	The name of the your computer.
CDPATH	echo $CDPATH	The search path for the cd command.
HISTFILE	echo $HISTFILE	The name of the file in which command history is saved.
HISTFILESIZE	echo $HISTFILESIZE	The maximum number of lines contained in the history file.
HISTSIZE	echo $HISTSIZE	The number of commands to remember in the command history. The default value is 500.
HOME	echo $HOME	The home directory of the current user.
IFS	echo $IFS	The Internal Field Separator that is used for word splitting after expansion and to split lines into words with the read builtin command. The default value is .
LANG	echo $LANG	Used to determine the locale category for any category not specifically selected with a variable starting with LC_.
PATH	echo $PATH	The search path for commands. It is a colon-separated list of directories in which the shell looks for commands.
PS1	echo $PS1	Your prompt settings.
TMOUT	echo $TMOUT	The default timeout for the read builtin command. Also in an interactive shell, the value is interpreted as the number of seconds to wait for input after issuing the command. If not input provided it will logout user.
TERM	echo $TERM	Your login terminal type.
SHELL	cho $SHELL	Set path to login shell.
DISPLAY	echo $DISPLAY	Set X display name
PWD	echo $PWD	current working directory
USERNAME	echo $USERNAME	Shell name

User Defined Variable

Shells also allow you to create your own variables for use within scripts (local variables) and to pass between scripts (global variables). User variables are traditionally created using lower-case characters.

To create a variable merely choose a lower-case name for the variable and give it a value using an equal (=) sign. Make certain that there are no spaces on either side of the equal sign. You can use the unset command to nullify the value of a previous set variable. Here are some examples.

Syntax

```
variable_name = value
$a = 10
echo $a
```

defined string variable in linux the following example

```
$company =' Prime Technology'
echo $company;
```

Shell Keywords

Keywords are the words whose meaning has already been explained to the shell. The keywords cannot be used as variable names because of it are reserved words with containing reserved meaning.

echo	read	set	unset
readonly	shift	export	if
fi	else	while	do
done	for	until	case
esac	break	continue	exit
return	trap	wait	eval
exec	ulimit	umask	

Command-Line Arguments?

Command-line arguments are parameters that are passed to a script while executing them in the bash shell.

They are also known as positional parameters in Linux.

How Shell Scripts Understand Command Line Arguments

Command-line arguments help make shell scripts interactive for the users. They help a script identify the data it needs to operate on. Hence, command-line arguments are an essential part of any practical shell scripting uses.

The shell has special variables reserved to point to the arguments which we pass through a shell script. Bash saves these variables numerically ($1, $2, $3, … $n)

Here, the first command-line argument in our shell script is $1, the second $2 and the third is $3. This goes on till the 9th argument. The variable $0 stores the name of the script or the command itself.

The special character $# stores the total number of arguments. We also have $@ and $* as wildcard characters which are used to denote all the arguments. We use $$ to find the process ID of the current shell script, while $? can be used to print the exit code for our script.

Read Command-line Arguments in Shell Scripts

Now we have developed an understanding of the command-line arguments in Linux. Now it's time to use this knowledge for practical application of the netstat command.

This is the shell script which we plan to use for this purpose.

```
echo "Script Name: $0"
echo "First Parameter of the script is $1"
echo "The second Parameter is $2"
echo "The complete list of arguments is $@"
echo "Total Number of Parameters: $#"
echo "The process ID is $$"
echo "Exit code for the script: $?"
```

Linux exit command

Linux exit command is used to exit from the current shell. It takes a parameter as a number and exits the shell with a return of status number. If we did not provide any parameter, it would return the status of the last executed command. The exit command closes a script and exits the shell.

Syntax:

```
exit
```

The exit command is the most useful common in Linux. We can pass with it many times. It uses the exit() function to terminate the normal process. Some points regarding the exit command are as following:

- It is used to exit from the shell script with an exit status N.
- It can be used to verify whether the shell script is successfully terminated or not.
- The exit status can be used by other commands to take their actions.
- If the value of N is not specified, the exit status will be the last executed command.
- The value of N is set to 0, and it stands for the normal shell exit.

Options:

The exit command does not provide many options. But, it supports the following options:

Exit without any parameter:

If we execute it without any parameter, it simply closes the terminal. Execute it as follows:

```
exit
```

The above command will simply close the terminal.

Exit with exit status:

If we pass a parameter, it will close and return the same exit status. For example, if we execute it with an exit status 10, it will return a status of 10. consider the below command:

```
exit 10
```

To display the exit status of the last executed command, execute the "echo $?" command as follows:

```
echo $?
```

Exit status of a shell script

Let's create a script, 'Demo.sh.' To create it, execute the following command:

1. cat > Demo.sh
2. echo " This is a demo."
3. exit 0

Save the above script by pressing the CTRL+D keys. Now, execute the script by executing the following commands:

1. chmod +x Demo.sh
2. ./Demo.sh

The above command will execute the script. Now, check the exit status of the script by executing the below command:

1. echo $?

The exit command is a built-in utility. However, it supports the "-help" option, which displays information about the command. To get help, execute the command as follows:

```
exit --help
help exit
```

Shell Scripting – Different types of Variables

The shell is a command-line interpreter for Linux and Unix systems. It provides an interface between the user and the kernel and executes commands. A sequence of commands can be written in a file for execution in the shell. It is called shell scripting. It helps to automate tasks in Linux.

In shell scripting there are three main types of variables are present. They are –

- Local Variables
- Global Variables or Environment Variables
- Shell Variables or System Variables

Local Variable

A local variable is a special type of variable which has its scope only within a specific function or block of code. Local variables can override the same variable name in the larger scope. Let's understand this concept using an example –

Shell script:

```
getName(){
NAME=SATYAJIT #local variable
echo "$NAME (from function)" #valid if called using function
}
echo "$NAME - (outside function)" #invalid here
getName
```

Output:

```
 - (outside function)
SATYAJIT (from function)
```

In this example, NAME is a local variable and its scope is limited within the getName() function.

Global Variables

A global variable is a variable with global scope. It is accessible throughout the program. Global variables are declared outside any block of code or function. Using an example –

```
NAME=SATYAJIT #global variable
getName(){
echo "$NAME (from function)"
}
echo "$NAME - (outside function)"
getName
```

Output:

```
SATYAJIT - (outside function)
SATYAJIT (from function)
```

Now, we have updated the earlier example code and declared the variable NAME outside any function. It makes the NAME variable a global variable that has its scope all over the program. Thus, it can be accessed from inside or outside a function.

Shell Variables

These are special types of variables. They are created and maintained by Linux Shell itself. These variables are required by the shell to function properly They are defined in Capital letters. Some useful shell variables are –

Variable Name	Description	Usage
BASH_VERSION	Holds the version of this instance of bash.	echo $BASH_VERSION
HOME	Provides a home directory of the current user.	echo $HOME
HOSTNAME	Provides computer name	echo $HOSTNAME
USERNAME	Provides username	echo $USERNAME

Shell Scripting – Rules for Naming Variable Name

Variables are quite important in any script or program, so we need to understand what is the convention to name these variables. There are certain rules and standards to keep in mind while giving names to the variables in Shell scripting.

Rule 1: A variable name can have letters, numbers, and underscores

Rule 2: There should be no whitespace on either side of the assignment operator(=)

The correct way to assign a variable in shell scripting is without any whitespace on either side of the assignment operator(=) which is shown as follows:
name="John"

Rule 3: The variable name cannot have special characters

Rule 4: The first character of the variable name cannot be a number
1st_name
6_gate

Rule 5: Variable names cannot be reserved words
if, elif, else , then, while

Rule 6: Variable name cannot have whitespace in between
first name="John"

Linux - Shell Basic Operators

There are various operators supported by each shell. We will now discuss the following operators −

- Arithmetic Operators
- Relational Operators
- Boolean Operators
- String Operators
- File Test Operators

Arithmetic Operators

The following arithmetic operators are supported by Bourne Shell.

Assume variable a holds 10 and variable b holds 20 then −

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator	`expr $a + $b` will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand	`expr $a - $b` will give -10
* (Multiplication)	Multiplies values on either side of the operator	`expr $a \* $b` will give 200
/ (Division)	Divides left hand operand by right hand operand	`expr $b / $a` will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder	`expr $b % $a` will give 0
= (Assignment)	Assigns right operand in left operand	a = $b would assign value of b into a
== (Equality)	Compares two numbers, if both are same then returns true.	[$a == $b] would return false.
!= (Not Equality)	Compares two numbers, if both are different then returns true.	[$a != $b] would return true.

It is very important to understand that all the conditional expressions should be inside square braces with spaces around them, for example [$a == $b] is correct whereas, [$a==$b] is incorrect.

.

Relational Operators

Bourne Shell supports the following relational operators that are specific to numeric values.

Assume variable a holds 10 and variable b holds 20 then −

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[$a -eq $b] is not true.
-ne	Checks if the value of two operands are equal or not; if values are not equal, then the condition becomes true.	[$a -ne $b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand; if yes, then the condition becomes true.	[$a -gt $b] is not true.
-lt	Checks if the value of left operand is less than the value of right operand; if yes, then the condition becomes true.	[$a -lt $b] is true.

-ge	Checks if the value of left operand is greater than or equal to the value of right operand; if yes, then the condition becomes true.	[$a -ge $b] is not true.
-le	Checks if the value of left operand is less than or equal to the value of right operand; if yes, then the condition becomes true.	[$a -le $b] is true.

It is very important to understand that all the conditional expressions should be placed inside square braces with spaces around them. For example, [$a <= $b] is correct whereas, [$a <= $b] is incorrect.

Boolean Operators

The following Boolean operators are supported by the Bourne Shell.
Assume variable a holds 10 and variable b holds 20 then −

Operator	Description	Example
!	This is logical negation. This inverts a true condition into false and vice versa.	[! false] is true.
-o	This is logical OR. If one of the operands is true, then the condition becomes true.	[$a -lt 20 -o $b -gt 100] is true.
-a	This is logical AND. If both the operands are true, then the condition becomes true otherwise false.	[$a -lt 20 -a $b -gt 100] is false.

String Operators

The following string operators are supported by Bourne Shell.
Assume variable a holds "abc" and variable b holds "efg" then −

Operator	Description	Example
=	Checks if the value of two operands are equal or not; if yes, then the condition becomes true.	[$a = $b] is not true.
!=	Checks if the value of two operands are equal or not; if values are not equal then the condition becomes true.	[$a != $b] is true.
-z	Checks if the given string operand size is zero; if it is zero length, then it returns true.	[-z $a] is not true.
-n	Checks if the given string operand size is non-zero; if it is nonzero length, then it returns true.	[-n $a] is not false.
str	Checks if str is not the empty string; if it is empty, then it returns false.	[$a] is not false.

File Test Operators

We have a few operators that can be used to test various properties associated with a Unix file.
Assume a variable file holds an existing file name "test" the size of which is 100 bytes and has read, write and execute permission on −

Operator	Description	Example
-b file	Checks if file is a block special file; if yes, then the condition becomes true.	[-b $file] is false.
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	[-c $file] is false.
-d file	Checks if file is a directory; if yes, then the condition becomes true.	[-d $file] is not true.
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true.	[-f $file] is true.
-g file	Checks if file has its set group ID (SGID) bit set; if yes, then the condition becomes true.	[-g $file] is false.
-k file	Checks if file has its sticky bit set; if yes, then the condition becomes true.	[-k $file] is false.
-p file	Checks if file is a named pipe; if yes, then the condition becomes true.	[-p $file] is false.
-t file	Checks if file descriptor is open and associated with a terminal; if yes, then the condition becomes true.	[-t $file] is false.
-u file	Checks if file has its Set User ID (SUID) bit set; if yes, then the condition becomes true.	[-u $file] is false.
-r file	Checks if file is readable; if yes, then the condition becomes true.	[-r $file] is true.
-w file	Checks if file is writable; if yes, then the condition becomes true.	[-w $file] is true.
-x file	Checks if file is executable; if yes, then the condition becomes true.	[-x $file] is true.
-s file	Checks if file has size greater than 0; if yes, then condition becomes true.	[-s $file] is true.
-e file	Checks if file exists; is true even if file is a directory but exists.	[-e $file] is true.

Conditional Statements: There are total 5 conditional statements which can be used in shell programming

1. **if statement**
2. **if-else statement**
3. **if..elif..else..fi statement (Else If ladder)**
4. **if..then..else..if..then..fi..fi..(Nested if)**
5. **switch statement**

Their description with syntax is as follows:

if statement

This block will process if specified condition is true.

Syntax:

if [expression]

```
then
  statement
fi
```

if-else statement

If specified condition is not true in if part then else part will be execute.

Syntax

```
if [ expression ]
then
  statement1
else
  statement2
fi
```

if..elif..else..fi statement (Else If ladder)

To use multiple conditions in one if-else block, then elif keyword is used in shell. If expression1 is true then it executes statement 1 and 2, and this process continues. If none of the condition is true then it processes else part.

Syntax

```
if [ expression1 ]
then
  statement1
  statement2
 .
elif [ expression2 ]
then
  statement3
  statement4.
else
  statement5
fi
```

if..then..else..if..then..fi..fi..(Nested if)

Nested if-else block can be used when, one condition is satisfies then it again checks another condition. In the syntax, if expression1 is false then it processes else part, and again expression2 will be check.

Syntax:

```
if [ expression1 ]
then
  statement1
  statement2
else
  if [ expression2 ]
  then
```

```
    statement3
    .
  fi
fi
```

switch statement

case statement works as a switch statement if specified value match with the pattern then it will execute a block of that particular pattern

When a match is found all of the associated statements until the double semicolon (;;) is executed.

A case will be terminated when the last command is executed.

If there is no match, the exit status of the case is zero.

Syntax:

```
case  in
  Pattern 1) Statement 1;;
  Pattern n) Statement n;;
esac
```

Looping Statements in Shell Scripting: There are total 3 looping statements that can be used in bash programming

1. **while statement**
2. **for statement**
3. **until statement**

To alter the flow of loop statements, two commands are used they are,

1. break
2. continue

Their descriptions and syntax are as follows:

while statement

Here the command is evaluated and based on the resulting loop will execute, if the command is raise to false then the loop will be terminated *that*

```
while <condition>
do

  <command 1>
  <command 2>
  <etc>
done
```

for statement

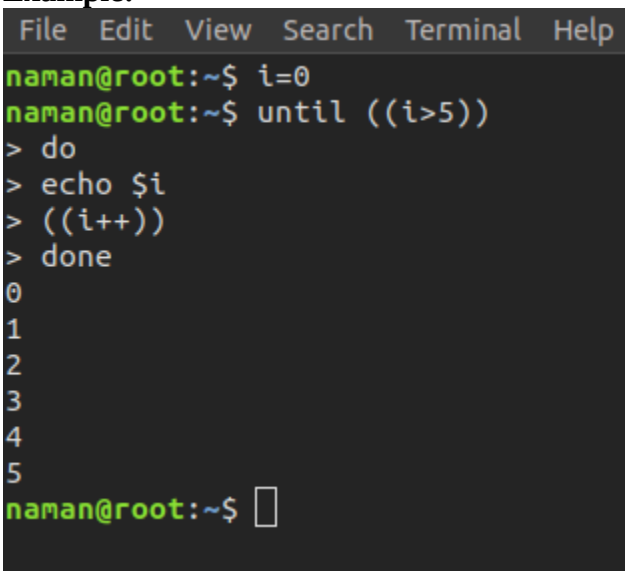
The for loop operates on lists of items. It repeats a set of commands for every item in a list. Here var is the name of a variable and word1 to wordN are sequences of characters separated by spaces (words). Each time the for loop executes, the value of the variable var is set to the next word in the list of words, word1 to wordN.

Syntax:
for <var> in <value1 value2 ... valuen>
do
   <command 1>
   <command 2>
   <etc>
done

until statement
The until loop is executed as many times as the condition/command evaluates too false. The loop terminates when the condition/command becomes true.
Syntax:
until <condition>
do

   <command 1>
   <command 2>
   <etc>
done

Example:



Shell Scripting – Set Command

Set command: It is used to set or unset specific flags and settings(determines the behavior of the script and helps in executing the tasks without any issue.) inside the shell environment. It can be used to change or display the shell attributes and parameters.

Syntax:

set -options arguments

The set command supports the following options:

Option	Description
-a	use to mark variables that are created or modified or created for export.
-b	use to notify the termination of the job.
-e	use to exit when the command exits with a non-zero status.
-f	it disables the file name generation known as globbing
-h	It saves the location of the command where it got looked.
-k	It places all assignment arguments in the environment variable of a command. Exception: It excludes all arguments that precede the command name.
-m	It enables job control
-n	It is used to read the commands.
-o	It is used for naming the option
-p	It disables the processing of the '$ENV' file and also imports the shell functions. Turned on: when the real and effective user ids do not match.
-t	It uses to exit from the command after executing a single command
-u	It treats unset variable as an error during substitution
-v	It prints the shell input lines
-x	It prints the commands and their arguments in the same sequence as they got executed.
-B	It performs the Brace expansion by the shell
-C	It disallows the existing regular files to be overwritten
-E	used when shell functions inherit the ERR trap
-H	It enables style history substitution. It is on by default.
-P	used when during command execution we don't want to follow the symbolic links.
-T	set this flag, this helps shell functions to inherit the DEBUG trap

For a demonstration of the use of the set command, let's use some set command.

Set -x command

This option prints the commands in the sequence as they got executed or is mainly used to do some script debugging..

Code:

```
set -x
echo Hello
```

echo Romy
Output:
bar
hello
Romy
+ echo bar
+ echo hello
+ echo Romy
We can see the commands getting printed after the command execution with the '+' sign.

Set -e command
It terminates the execution when the error occurs.
Code:(without set -e)
echo Hello
foo
echo Romy
Output:
Hello
Romy
main.sh: line 14: foo: command not found
'foo' is a non-existent command but bash still executed the third line after encountering the error at the second line. We can use the set command to stop termination.

Piped commands
Set -e command does not work with piped commands.
Example:
set -e
foo | echo " This is the piped command"
echo "executed"
Output:
This is the piped command
executed

Unset positional parameters
In order to unset the positional parameter, run the set command followed by two hyphens(set –).
Example:
set apple mango orange guava

set --
echo $1
echo $2

echo "Hello"
Output:
Hello
We can see that nothing is getting printed corresponding to the first two commands to print positional parameters.

Shift command
Shift is a built-in command in bash which after getting executed, shifts/move the command line arguments to one position left. The first argument is lost after using shift command. This command takes only one integer as an argument. This command is useful when you want to get rid of the command line arguments which are not needed after parsing them.
Syntax:
shift n
Here, n is the number of positions by which you want to shift command-line arguments to the.

unset command
The unset command is a shell command that clears the value of an exported environment variable. This means that if you run unset EXPORTED_VARAIBLE, then any process will no longer have access to the value of the variable. If you want to remove the variable completely, you need to run unset -f EXPORTED_VARIABLE.

$ export file_name="test.txt"
$ env | grep file_name
file_name=test.txt
$ unset file_name
$ env | grep file_name

We're verifying that the variable has been removed from our shell environment by running the env command.

Assigning the Empty Value
You can clear the values of exported variables by assigning them empty values.
$ export file_name="test.txt"
$ env | grep file_name
file_name=test.txt
$ export file_name=
$ env | grep file_name

Using the *env* Command

The env command lets us run programs in a modified environment. We can then use this command to reset the export status of an exported variable.

```
$ export file_name="test.txt"
$ env | grep file_name
file_name=test.txt
$ env -i bash
$ env | grep file_name
```

Accessing Environment Variables

In Linux, the primary conduit for interacting with environment variables is the shell. The shell acts as a command-line interpreter, executing instructions entered by the user.

Scope of an environment variable

Understanding the scope of an environment variable is crucial. It dictates where the variable can be accessed or defined, making a clear distinction between global and local scopes.

Global Environment Variables

A global environment variable, when defined in a terminal, enjoys accessibility from anywhere within that terminal's environment. This means it can be employed by scripts, programs, or processes running within the scope of that terminal.

Local Environment Variables

On the other hand, local environment variables are confined to the specific terminal in which they are defined.

Accessing Environment Variables

Accessing environment variables is straightforward and consistent, regardless of their scope. To retrieve the value of an environment variable, the following syntax is employed:

Syntax

```
VARIABLE_NAME
```

This syntax is uniform for both local and global environment variables.

Displaying Environment Variables

To display the value of an environment variable, the `echo` command is employed, as demonstrated below:

Syntax:

```
echo $VARIABLE_NAME
```

Alias in Linux

In Linux, **an alias is a shortcut** that references a command. An alias replaces a string that invokes a command in the Linux shell with another user-defined string.

Aliases are mostly used to replace long commands, improving efficiency and avoiding potential spelling errors. Aliases can also replace commands with additional options, making them easier to use.

Linux Alias Syntax

The **`alias`** command uses the following syntax:

```
alias [option] [name]='[value]'
```

The different elements of the **`alias`** command syntax are:

- **`alias`**: Invokes the **`alias`** command.
- **`[option]`**: Allows the command to list all current aliases.
- **`[name]`**: Defines the new shortcut that references a command. A name is a user-defined string, excluding special characters and **'alias'** and **'unalias'**, which cannot be used as names.
- **`[value]`**: Specifies the command the alias references. Commands can also include options, arguments, and variables. A value can also be a path to a script you want to execute.

Note: Enclosing the value in single quotation marks (') **will not expand** any variables used with the command. To expand the variables, use double quotation marks (").

Create Aliases in Linux

There are two types of aliases to create in Linux:

- **Temporary**. Add them using the **`alias`** command.
- **Permanent**. These require to edit system files.

Create a Temporary Alias in Linux

Use the **`alias`** command to create a temporary alias that lasts until the end of the current terminal session. For instance, creating **`c`** as an alias for the clear command:

```
alias c='clear'
```

Example 1:
Implementing if statement

```
#Initializing two variables
a=10
b=20

#Check whether they are equal
if [ $a == $b ]
then
   echo "a is equal to b"
fi
#Check whether they are not equal
if [ $a != $b ]
then
   echo "a is not equal to b"
fi
```

Output

```
$bash -f main.sh
a is not equal to b
```

Example 2:
Implementing if.else statement

```
#Initializing two variables
a=20
b=20

if [ $a == $b ]
then
   #If they are equal then print this
   echo "a is equal to b"
else
   #else print this
   echo "a is not equal to b"
fi
```

Output

```
$bash -f main.sh
a is equal to b
```

Example 3:
Implementing switch statement

```
CARS="bmw"

#Pass the variable in string
```

```
case "$CARS" in
   #case 1
   "mercedes") echo "Headquarters - Affalterbach, Germany" ;;

   #case 2
   "audi") echo "Headquarters - Ingolstadt, Germany" ;;

   #case 3
   "bmw") echo "Headquarters - Chennai, Tamil Nadu, India" ;;
esac
```

Output

```
$bash -f main.sh
Headquarters - Chennai, Tamil Nadu, India.
```

Note: Shell scripting is a case-sensitive language, which means proper syntax has to be followed while writing the scripts.

```
# HOW TO FIND A NUMBER IS EVEN OR ODD IN SHELL SCRIPT
clear
echo "---- EVEN OR ODD IN SHELL SCRIPT -----";
echo -n "Enter a number:";
read n;
echo -n "RESULT: ";
if [ `expr $n % 2` == 0 ]
then
        echo "$n is even";
else
        echo "$n is Odd";
fi

#Leap Year Program with  Shell Scripting
echo -n "Enter year (YYYY): "
read y
a = 'expr $y%4'
b = 'expr $y%100'
c = 'expr $y%400'
if [$a -eq 0 -a $b -ne 0 -o $c -eq 0]
then
echo "$y is leap year"
else
echo "$y is not a leap year"
```

```
fi

Shell Script to print odd and even numbers
# Take user input
echo "Enter a number"
read n

echo "Even Numbers - "
i=1

# -le means less than or equal to
while [ $i -le $n ]
do
# arithmetic operations is performed with 'expr'
rs=`expr $i % 2`

if [ $rs == 0 ]
then
echo "  $i"

# end of if loop
fi

# incrementing i by one
((++i))

# end of while loop
done

# Now printing odd numbers
echo "Odd Numbers - "
i=1

while [ $i -le $n ]
do

rs=`expr $i % 2`

if [ $rs != 0 ]
then
```

```
echo "  $i"

fi

((++i))
done
```

Output:

```
Enter a number
10
Even Numbers -
  2
  4
  6
  8
  10
Odd Numbers -
  1
  3
  5
  7
  9
```

Shell Script to print first n natural numbers

```
cho "Enter a integer number"
read n
a=1
echo "Printing numbers:"
while [ $a -le $n ]
do
echo "$a"
a=`expr $a + 1`
done
```

Output

```
Enter a integer number
6
Printing numbers:
```

1
2
3
4
5
6

Implementing for loop with break statement

```
#Start of for loop
for a in 1 2 3 4 5 6 7 8 9 10
do
   # if a is equal to 5 break the loop
   if [ $a == 5 ]
   then
      break
   fi
   # Print the value
   echo "Iteration no $a"
done
```

Output:

```
$bash -f main.sh
Iteration no 1
Iteration no 2
Iteration no 3
Iteration no 4
```

set Command

We use the *set* command to change the values of shell options and display variables in Bash scripts. We can also use it to debug Bash scripts, export values from shell scripts, terminate programs when they fail, and handle exceptions.

The *set* command has the following format:

```
$ set [option] [argument]
```

Here, we use *option* to either set or unset a flag while *argument* is a positional parameter. The o*ption* flag impacts how Bash scripts behave to accomplish any desired task.

When we run the *set* command without any arguments, it returns a list of all system settings. The list includes the names and values of all shell variables and functions:

set Command Options

We have several options we can use with the *set* command.

The *-e* Option

When a query returns a non-zero status, the *-e* flag stops the script. It also detects errors in the currently executing script.

Let's write a script that displays the contents of a file and outputs a message if the file exists in our directory:

```
set -e
mkdir newfolder
cat filenotindirectory
echo 'The file is not in our directory!'
```

We run the executable file to test out the *-e* flag:

```
$ chmod u+x eflag.sh
$ ./eflag.sh
cat: can't open 'filenotindirectory': No such file or directory
```

When the script tries to display the contents of the file *filenotindirectory*, it errors out because the file does not exist, and it exits immediately. As a result, the final *echo* command is also not run.

The *-C* Option

Using **the *-C* flag ensures that we cannot overwrite an existing file with the same name**:

```
$ echo 'An existing file' > myfile
$ set -C
$ echo 'Editing an existing file' > myfile
sh: can't create myfile: File exists
```

We created a file called *myfile*, but an error occurred when we tried to overwrite its contents. Using this flag is a good practice to avoid accidental overwriting of files.

The *-f* Option

Searching for files with the *.txt* extension:

```
$ ls *.txt
files.txt
```

We can see that there is one file with the *.txt* extension.

Next, let's run the *set -f* command and then search for files with the *.txt* extension using a wildcard:

```
$ set -f
$ ls *.txt
ls: *.txt: No such file or directory
```

We now get an error as output. **The *-f* prevents us from using wildcards to search for** filenames or strings.

We should note that running the ***ls*** command without specifying any wildcard pattern would still give an output of all the files in our directories**.**

The *-x* Option

We use the ***-x*** parameter when debugging our scripts to determine the output of individual commands.

To create a script that displays a countdown from 3 to 0:

```
#!/bin/bash
set -x
n=3
while [ $n -gt 0 ]; do
    n=$[ $n-1 ]
    echo $n
    sleep 1
done
```

\

Shift command

Shift is a builtin command in bash which after getting executed, shifts/move the command line arguments to one position left. The first argument is lost after using shift command. This command takes only one integer as an argument. This command is useful when you want to get rid of the command line arguments which are not needed after parsing them.

Syntax:

```
shift n
```

Here, n is the number of positions by which you want to shift command-line arguments to the left if you do not specify, the default value of n is assumed to be 1 i.e shift works the same as shift 1.

Example: Let's create a shell script file named as *sampleshift.sh* as follows. The total number of command-line arguments is represented by $#. Use the following command to create the desired shell script file

```
vi sampleshift.sh
```

Now paste the following code:

```
#!/bin/bash

# total number of command-line arguments
echo "Total arguments passed are: $#"

# $* is used to show the command line arguments
echo "The arguments are: $*"

echo "The First Argument is: $1"
shift 2

echo "The First Argument After Shift 2 is: $1"
shift

echo "The First Argument After Shift is: $1"
```

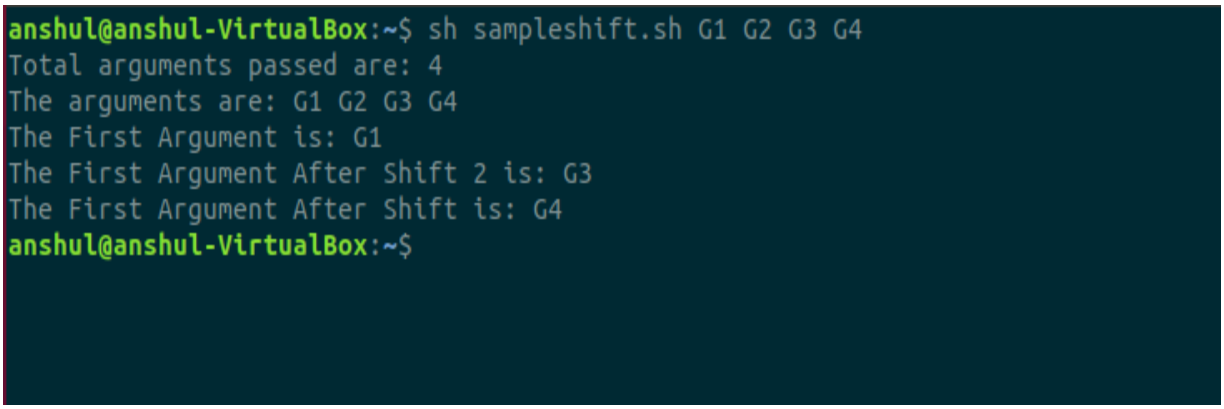
Now to save the file press ESC and then type **":x"** without quotes and hit Enter. Now to execute the file, use the following command on Linux terminal

```
sh sampleshift.sh
```

But here we have to pass command-line arguments so we can use the following command

```
sh sampleshift.sh G1 G2 G3 G4
```

Here, we are passing 4 command-line arguments named G1, G2, G3, and G4. Below is the screenshot of the output of using shift command:



Expanding Null or Unset Variables

The expression ${name} (or just $name if it is not ambiguous) expands to the value of the name variable (If name is null or not set, the shell expands ${Name} to a null string. The shell provides the following alternatives to accepting the expanded null string as the value of the variable.

- Use a default value for the variable.
- Use a default value and assign that value to the variable.
- Display an error.

A variable is a character string to which we assign a value. The value assigned could be a number, text, filename, device, or any other type of data.
A variable is nothing more than a pointer to the actual data. The shell enables you to create, assign, and delete variables.

Variable Names

The name of a variable can contain only letters (a to z or A to Z), numbers (0 to 9) or the underscore character (_).
By convention, Unix shell variables will have their names in UPPERCASE.
The following examples are valid variable names −

```
_ALI
TOKEN_A
VAR_1
VAR_2
```

Defining Variables

Variables are defined as follows −

```
variable_name=variable_value
```

For example −

```
NAME="Mitrc"
```

Shell enables you to store any value you want in a variable. For example −

```
VAR1="Linux"
VAR2=100
```

Accessing Values

To access the value stored in a variable, prefix its name with the dollar sign (**$**) −
For example, the following script will access the value of defined variable NAME and print it on STDOUT −

```
NAME="Linux"
echo $NAME
```

The above script will produce the following value −

```
Linux
```

Read-only Variables

Shell provides a way to mark variables as read-only by using the read-only command. After a variable is marked read-only, its value cannot be changed.
For example, the following script generates an error while trying to change the value of NAME −

```
NAME="Linux"
readonly NAME
NAME="abx"
```

The above script will generate the following result −

```
/bin/sh: NAME: This variable is read only.
```

Unsetting Variables

Unsetting or deleting a variable directs the shell to remove the variable from the list of variables that it tracks. Once you unset a variable, you cannot access the stored value in the variable.

Following is the syntax to unset a defined variable using the **unset** command −

```
unset variable_name
```

The above command unsets the value of a defined variable. Here is a simple example that demonstrates how the command works −

```
#!/bin/sh

NAME="Linux"
unset NAME
echo $NAME
```

The above example does not print anything. You cannot use the unset command to **unset** variables that are marked **readonly**.

Linux aliases

Linux 'alias' command replaces one string from the shell with another string. It is a shell built-in command. It converts a complicated command into a simpler command or in other words, it creates a shortcut by replacing it with the simpler one.

Linux Alias Syntax

The alias command uses the following syntax:

alias [option] [name]='[value]'

The different elements of the alias command syntax are:

- alias: Invokes the alias command.
- [option]: Allows the command to list all current aliases.
- [name]: Defines the new shortcut that references a command. A name is a user-defined string, excluding special characters and 'alias' and 'unalias', which cannot be used as names.
- [value]: Specifies the command the alias references. Commands can also include options, arguments, and variables. A value can also be a path to a script you want to execute.

Note: Enclosing the value in single quotation marks (') will not expand any variables used with the command. To expand the variables, use double quotation marks (").

Creating an alias

Here, we are going to use following options for creating an alias.

1. Creating alias for 'file' command as 'fi'
2. Creating alias for 'ls-l' command as 'll'
3. Creating alias with two arguments

4. Creating alias for a path

1) Creating alias for 'file' command as 'fi'

Syntax:

1. alias <newName>=<command>

Example:

1. alias fi=file

Look at the above snapshot, 'file' command is aliased as 'fi' through command "alias fi=file".

2) Creating alias for 'ls-l' command as 'll'

Syntax:

1. alias <newName>=<'command'>

Example:

1. alias ll='ls -l'

Look at the above snapshot, 'ls -l' command is aliased as 'll' through command "alias ll='ls -l' ".

3) Creating alias with two arguments

Syntax:

1. alias <newName>=<'command arg1 arg2'>

Example:

1. alias both='ls sample example'

Look at the above snapshot, 'ls sample example' command is aliased as 'both' through command "alias both='ls sample example' ".

4) Creating alias for a path

Syntax:

1. alias <newName>=<'/home/sssit/path/...'>

Example:

1. alias path='cd /home/sssit/Downloads/sample'

Look at the above snapshot, 'cd /home/sssit/Downloads/sample' command is aliased as 'path' through command "alias path='cd /home/sssit/Downloads/sample' ".

How to remove alias

With the help of 'unalias' command you can remove created alias.

Syntax:

1. unalias <createdAlias>

Example:

1. unalias c

Create Aliases in Linux

There are two types of aliases to create in Linux:

- Temporary. Add them using the alias command.
- Permanent. These require to edit system files.

Create a Temporary Alias in Linux

Use the alias command to create a temporary alias that lasts until the end of the current terminal session. For instance, creating c as an alias for the clear command:

```
alias c='clear'
```

Note: The alias command allows you to include multiple commands as the value by dividing them with the pipe symbol (|).

Remove Aliases in Linux

To remove an alias, use the unalias command with the following syntax:

```
unalias [name]
```

For instance, to remove the frename alias:

```
unalias frename
```

Adding the -a option allows you to remove all aliases:

```
unalias -a
```

Linux - Shell Functions

Using functions to perform repetitive tasks is an excellent way to create code reuse. This is an important part of modern object-oriented programming principles.

Shell functions are similar to subroutines, procedures, and functions in other programming languages.

Creating Functions

To declare a function, simply use the following syntax −

```
function_name () {
   list of commands
}
```

The name of your function is function_name, and that's what you will use to call it from elsewhere in your scripts. The function name must be followed by parentheses, followed by a list of commands enclosed within braces.

Following example shows the use of function −

```
# Define your function here
Hello () {
   echo "Hello World"
}
```

Invoke your function
Hello
Upon execution, you will receive the following output −
$./test.sh
Hello World
Pass Parameters to a Function
You can define a function that will accept parameters while calling the function. These parameters would be represented by $1, $2 and so on.

Define your function here
```
Hello () {
  echo "Hello World $1 $2"
}

# Invoke your function
Hello Zara Ali
```
Upon execution, you will receive the following result −
$./test.sh
Hello World Zara Ali

Write shell script to print first 10 natural numbers

```
a=0

while [ $a -lt 10 ]
do
   echo $a
   a=`expr $a + 1`
done
```

Upon execution, you will receive the following result −

```
0
1
2
3
4
5
6
7
8
9
```

Write a shell script to find the factorial of number?

```
read -p "Enter a number" num
fact=1

for((i=2;i<=num;i++))
{
  fact=$((fact*i))
}

echo $fact
```

A shell script to calculate the factorial of input number using while loop.

```
# A shell script to find the factorial of a number

read -p "Enter a number" num
fact=1
```

```
while [ $num -gt 1 ]
do
  fact=$((fact*num))
  num=$((num-1))
done

echo $fact
```

Example: Using break with the value of N. Consider an example:

```
for i in `seq 1 5`
 do
  for j in `seq 1 5`
       do
            if(( $j== 2 ))
                 then
                 break 2
            fi
            echo "value of j is $j"
       done
  echo "value of i is $i"
done
```

As the value of break is 2, when the value of j is 2, both the loops are exited in a single step. So, the only value of j is printed when it was 1.

```
value of j is 1
```

continue

Continue is a command which is used to skip the remaining command inside the loop for the current iteration in for, while, and until loop.

Syntax:

```
continue [N]
// the optional parameter N specifies the nth enclosing loop to continue from.
// This parameter is optional.
// By default the value of N is 1.
```

Using break command in a loop

```
[mishra@localhost ~]$ for i in `seq 1 5`
> do
> if(( $i==3 ))
> then
> continue
> fi
> echo $i
> done
1
2
4
5
```

Example: Using continue with the value of N. Consider an example:

```
for i in `seq 1 5`
 do
  for j in `seq 1 5`
       do
            if(( $j== 2 ))
            then
                 continue 2
            fi
            echo "value of j is $j"
       done
  echo "value of i is $i"
done
```

Continue skips both the loop when the value of j is 2 and hence, the code only executes when the value of j is 1.

```
value of j is 1
value of j is 1
value of j is 1
value of j is 1
value of j is 1
```

Understanding Arrays in Shell Scripting

An array is a structured arrangement of similar data elements. Within shell scripting, an array is a variable that holds multiple values, whether they are of the same type or different types. It's important to note that in shell scripting, everything is treated as a string. Arrays adhere to a zero-based index, which signifies that indexing initiates from 0.

How to Declare Array in Shell Scripting?

Arrays can be declared in a shell script using various approaches:

1. Indirect Declaration

In this method, you assign a value to a specific index of the array variable. There's no need to declare the array beforehand.

```
ARRAYNAME[INDEXNR]=value
```

2. Explicit Declaration

With explicit declaration, you first declare the array and then assign values to it.

```
declare -a ARRAYNAME
```

3. Compound Assignment

This method involves declaring the array along with its initial set of values. You can later add additional values to the array.

```
ARRAYNAME=(value1 value2  .... valueN)
```

Alternatively, you can use index numbers to assign values explicitly:

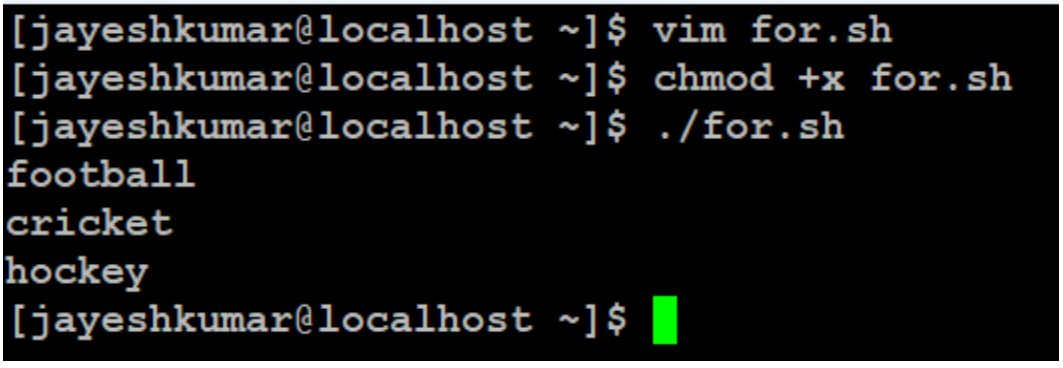
```
ARRAYNAME=([1]=10 [2]=20 [3]=30)
```

Array iteration for loops

We can iterate over arrays conveniently in bash using for loops with a specific syntax. We can use the special variables in BASH i.e @ to access all the elements in the array. Let's look at the code:

```
#!/bin/bash

s=("football" "cricket" "hockey")
for n in ${s[@]};
do
   echo $n
done
```



```
n=7
for (( i=1 ; i<=$n ; i++ ));
do
   echo $i
done
```

```
[jayeshkumar@localhost ~]$ vim for.sh
[jayeshkumar@localhost ~]$ chmod +x for.sh
[jayeshkumar@localhost ~]$ ./for.sh
1
2
3
4
5
6
7
[jayeshkumar@localhost ~]$
```

Sorting the array in Bash
using Bubble sort

```
# Static input of Array
arr=(10 8 20 100 12)

echo "Array in original order"
echo ${arr[*]}

# Performing Bubble sort
for ((i = 0; i<5; i++))
do

   for((j = 0; j<5-i-1; j++))
   do

      if [ ${arr[j]} -gt ${arr[$((j+1))]} ]
      then
         # swap
         temp=${arr[j]}
         arr[$j]=${arr[$((j+1))]}
         arr[$((j+1))]=$temp
      fi
   done
done

echo "Array in sorted order :"
```

```
echo ${arr[*]}
```

Output :

```
Array in sorted order :
8 10 12 20 100
```

```
n=5
arr=(10 8 20 100 12)

echo "Original array is: ${arr[*]}";

flag=1;
for (( i = 0; i < $n-1; i++ ))
do
   flag=0;
   for ((j = 0; j < $n-1-$i; j++ ))
   do
      if [[ ${arr[$j]} -gt ${arr[$j+1]} ]]
      then
         temp=${arr[$j]};
         arr[$j]=${arr[$j+1]};
         arr[$j+1]=$temp;
         flag=1;
      fi
   done

   if [[ $flag -eq 0 ]]; then
        break;
   fi
```

Output:

```
Original array is: 10 8 20 100 12
Final sorted Array is 8 10 12 20 100
```